

Aplicações de Busca Binária e Soma de Prefixos

Escaneie para acessar o slide da Aula 5



luisenilton.github.io/Curso-Maratona/aula5

Sumário da Aula

- | | |
|--|--------------------------------------|
| 1. Aplicações de Busca Binária e Soma de Prefixos. | 12. Soma de prefixos e Busca binária |
| 2. Sumário da Aula | 13. Problema clássico |
| 3. Função Monotônica | 14. Soma num Intervalo |
| 4. Questão motivadora | 15. Força bruta |
| 5. Função Booleana | 16. Array de diferenças |
| 6. Busca binária na resposta | 17. Problema |
| 7. Solução | 18. Referências |
| 8. Soma de prefixos | 19. Contatos |
| 9. Força bruta | 20. Obrigado por acompanhar a aula |
| 10. Problema Simplificado | |
| 11. Voltando ao problema original | |

Função Monotônica

Função Monotônica

- A busca binária não é restrita a problemas de encontrar valores no array ou primeiro valor $\geq$, na verdade ela pode ser aplicada em qualquer função definida como monotônica em $O(\log_2 n)$, mas o que é uma função monotônica?

Função Monotônica

- A busca binária não é restrita a problemas de encontrar valores no array ou primeiro valor $\geq$, na verdade ela pode ser aplicada em qualquer função definida como monotônica em $O(\log_2 n)$, mas o que é uma função monotônica?
- Lembre que em cada passo da busca binária nós precisamos cortar uma das metades do espaço de busca, então temos que ter certeza que o valor que buscamos não está daquele lado.

Função Monotônica

- A busca binária não é restrita a problemas de encontrar valores no array ou primeiro valor $\geq$, na verdade ela pode ser aplicada em qualquer função definida como monotônica em $O(\log_2 n)$, mas o que é uma função monotônica?
- Lembre que em cada passo da busca binária nós precisamos cortar uma das metades do espaço de busca, então temos que ter certeza que o valor que buscamos não está daquele lado.
- Uma função monotônica é aquela que mantém um único sentido de variação em todo o seu domínio. Isso significa que ela é sempre não crescente ou sempre não decrescente.

Função Monotônica

- A busca binária não é restrita a problemas de encontrar valores no array ou primeiro valor $\geq$, na verdade ela pode ser aplicada em qualquer função definida como monotônica em $O(\log_2 n)$, mas o que é uma função monotônica?
- Lembre que em cada passo da busca binária nós precisamos cortar uma das metades do espaço de busca, então temos que ter certeza que o valor que buscamos não está daquele lado.
- Uma função monotônica é aquela que mantém um único sentido de variação em todo o seu domínio. Isso significa que ela é sempre não crescente ou sempre não decrescente.
- Matematicamente nós temos 2 opções para uma função monotônica:
 - $x_2 > x_1 \ \&\& \ f(x_2) \geq f(x_1)$, não decrescente
 - $x_2 > x_1 \ \&\& \ f(x_2) \leq f(x_1)$, não crescente

Função Monotônica

- A busca binária não é restrita a problemas de encontrar valores no array ou primeiro valor $\geq$, na verdade ela pode ser aplicada em qualquer função definida como monotônica em $O(\log_2 n)$, mas o que é uma função monotônica?
- Lembre que em cada passo da busca binária nós precisamos cortar uma das metades do espaço de busca, então temos que ter certeza que o valor que buscamos não está daquele lado.
- Uma função monotônica é aquela que mantém um único sentido de variação em todo o seu domínio. Isso significa que ela é sempre não crescente ou sempre não decrescente.
- Matematicamente nós temos 2 opções para uma função monotônica:
 - $x_2 > x_1 \ \&\& \ f(x_2) \geq f(x_1)$, não decrescente
 - $x_2 > x_1 \ \&\& \ f(x_2) \leq f(x_1)$, não crescente
- Perceba que dado que estamos procurando um valor y , se chutarmos um valor x nós podemos comparar o valor de y com $f(x)$ e decidir para que lado ir.

Questão motivadora

Questão motivadora

Link

Questão motivadora

Link

<https://www.desmos.com/calculator?lang=pt-BR>

Questão motivadora

Link

<https://www.desmos.com/calculator?lang=pt-BR>

```
double l = 1, r = 1e10;

while((l - r) ≤ EPS){
    int x = (l + r)/2;
    // se o valor que chutamos está depois de C, temos que voltar
    if(f(x) > C){
        r = x;
    }else{
        l = x;
    }
}

cout << l << endl;
```

Função Booleana

Função Booleana

Uma função booleana retorna 'Verdadeiro' ou 'Falso' para qualquer valor X de entrada, vamos denotar Verdadeiro por 1 e Falso por 0 .

Função Booleana

Uma função booleana retorna 'Verdadeiro' ou 'Falso' para qualquer valor X de entrada, vamos denotar Verdadeiro por 1 e Falso por 0 .

Exemplo de função: Consigo levantar esse peso no supino?

Função Booleana

Uma função booleana retorna 'Verdadeiro' ou 'Falso' para qualquer valor X de entrada, vamos denotar Verdadeiro por 1 e Falso por 0 .

Exemplo de função: Consigo levantar esse peso no supino?

Peso X	5	10	15	20	25	30	35
Consigo? $f(X)$	1	1	1	0	0	0	0

Função Booleana

Uma função booleana retorna 'Verdadeiro' ou 'Falso' para qualquer valor X de entrada, vamos denotar Verdadeiro por 1 e Falso por 0 .

Exemplo de função: Consigo levantar esse peso no supino?

Peso X	5	10	15	20	25	30	35
Consigo? $f(X)$	1	1	1	0	0	0	0

Perceba que essa função é monotônica, se eu não consigo com um peso X , eu não conseguirei com $X + 1$. Logo, poderíamos fazer uma busca binária para determinar o maior peso que uma pessoa levanta no supino.

Busca binária na resposta

Busca binária na resposta

Problema: Uma fábrica tem n máquinas que podem ser usadas para fazer produtos. Seu objetivo é fazer um total de m produtos.

Busca binária na resposta

Problema: Uma fábrica tem n máquinas que podem ser usadas para fazer produtos. Seu objetivo é fazer um total de m produtos.

Para cada máquina, você sabe o número de segundos que ela precisa para fazer um único produto. As máquinas podem trabalhar simultaneamente, e você pode decidir livremente o cronograma delas.

Busca binária na resposta

Problema: Uma fábrica tem n máquinas que podem ser usadas para fazer produtos. Seu objetivo é fazer um total de m produtos.

Para cada máquina, você sabe o número de segundos que ela precisa para fazer um único produto. As máquinas podem trabalhar simultaneamente, e você pode decidir livremente o cronograma delas.

Qual é o tempo mais curto necessário para fazer m produtos?

Busca binária na resposta

Problema: Uma fábrica tem n máquinas que podem ser usadas para fazer produtos. Seu objetivo é fazer um total de m produtos.

Para cada máquina, você sabe o número de segundos que ela precisa para fazer um único produto. As máquinas podem trabalhar simultaneamente, e você pode decidir livremente o cronograma delas.

Qual é o tempo mais curto necessário para fazer m produtos?

Link

Busca binária na resposta

Problema: Uma fábrica tem n máquinas que podem ser usadas para fazer produtos. Seu objetivo é fazer um total de m produtos.

Para cada máquina, você sabe o número de segundos que ela precisa para fazer um único produto. As máquinas podem trabalhar simultaneamente, e você pode decidir livremente o cronograma delas.

Qual é o tempo mais curto necessário para fazer m produtos?

Link

Ao analisarmos um problema de mínimo nós podemos fazer a seguinte pergunta, se eu consigo resolver o problema com X eu também consigo resolver com $X + 1$?

Busca binária na resposta

Problema: Uma fábrica tem n máquinas que podem ser usadas para fazer produtos. Seu objetivo é fazer um total de m produtos.

Para cada máquina, você sabe o número de segundos que ela precisa para fazer um único produto. As máquinas podem trabalhar simultaneamente, e você pode decidir livremente o cronograma delas.

Qual é o tempo mais curto necessário para fazer m produtos?

Link

Ao analisarmos um problema de mínimo nós podemos fazer a seguinte pergunta, se eu consigo resolver o problema com X eu também consigo resolver com $X + 1$?

Se isso for verdade podemos modelar a função de resposta como uma função booleana que será monotônica com base na observação anterior, e aplicar busca binária.

Solução

Solução

Dado que a função de resposta monotônica, nós precisamos apenas criar uma função $\text{can}(t)$ que verifica se eu consigo resolver o problema com um tempo t , se for possível nós tentamos diminuir a resposta.

Solução

Dado que a função de resposta monotônica, nós precisamos apenas criar uma função $\text{can}(t)$ que verifica se eu consigo resolver o problema com um tempo t , se for possível nós tentamos diminuir a resposta.

É importante definirmos com precisão qual o menor e o maior valor possível para a resposta.

Solução

Dado que a função de resposta monotônica, nós precisamos apenas criar uma função `can(t)` que verifica se eu consigo resolver o problema com um tempo `t`, se for possível nós tentamos diminuir a resposta.

É importante definirmos com precisão qual o menor e o maior valor possível para a resposta.

```
int l = 0, r = 1e18;
int ans;
while(l ≤ r){
    int t = (l + r)/2;
    // se eu consigo com esse valor tento com menos
    if(can(t)){
        ans = t;
        r = t - 1;
    }else{
        l = t + 1;
    }
}
```

Solução

Dado que a função de resposta monotônica, nós precisamos apenas criar uma função $\text{can}(t)$ que verifica se eu consigo resolver o problema com um tempo t , se for possível nós tentamos diminuir a resposta.

É importante definirmos com precisão qual o menor e o maior valor possível para a resposta.

```
int l = 0, r = 1e18;
int ans;
while(l ≤ r){
    int t = (l + r)/2;
    // se eu consigo com esse valor tento com menos
    if(can(t)){
        ans = t;
        r = t - 1;
    }else{
        l = t + 1;
    }
}
```

Para checkar se é possível com um tempo t , podemos calcular quantos produtos cada máquina produz em t segundos e vemos se o total é maior do que m .

Soma de prefixos

Soma de prefixos

Problema Motivador

Soma de prefixos

Problema Motivador

Dado um array com $N (\leq 10^5)$ inteiros, responda $Q (\leq 10^5)$ consultas na forma:

Soma de prefixos

Problema Motivador

Dado um array com $N (\leq 10^5)$ inteiros, responda $Q (\leq 10^5)$ consultas na forma:

"Qual a soma dos valores no intervalo incluso $[a, b]$?"

Soma de prefixos

Problema Motivador

Dado um array com $N (\leq 10^5)$ inteiros, responda $Q (\leq 10^5)$ consultas na forma:

"Qual a soma dos valores no intervalo incluso $[a, b]$?"

Exemplo prático

Soma de prefixos

Problema Motivador

Dado um array com $N (\leq 10^5)$ inteiros, responda $Q (\leq 10^5)$ consultas na forma:

"Qual a soma dos valores no intervalo incluso $[a, b]$?"

Exemplo prático

Índice (i)	1	2	3	4	5	6	7	8
Valor	3	2	4	5	1	1	5	3

Soma de prefixos

Problema Motivador

Dado um array com $N (\leq 10^5)$ inteiros, responda $Q (\leq 10^5)$ consultas na forma:

"Qual a soma dos valores no intervalo incluso $[a, b]$?"

Exemplo prático

Índice (i)	1	2	3	4	5	6	7	8
Valor	3	2	4	5	1	1	5	3

Consultas

Soma de prefixos

Problema Motivador

Dado um array com N ($\leq 10^5$) inteiros, responda Q ($\leq 10^5$) consultas na forma:

"Qual a soma dos valores no intervalo incluso $[a, b]$?"

Exemplo prático

Índice (i)	1	2	3	4	5	6	7	8
Valor	3	2	4	5	1	1	5	3

Consultas

- $[2, 4] \rightarrow 2 + 4 + 5 = 11$
- $[5, 6] \rightarrow 1 + 1 = 2$

Força bruta

Força bruta

A primeira abordagem a se pensar é, porque não fazer um loop de $[l, r]$ somando o valor de cada elemento de cada elemento.

Força bruta

A primeira abordagem a se pensar é, porque não fazer um loop de $[l, r]$ somando o valor de cada elemento de cada elemento.

```
while(q--){  
    int l,r; cin >> l >> r;  
    int sum = 0;  
    for(int i = l; i ≤ r;i++) sum += a[i];  
    cout << sum << endl;  
}
```

Força bruta

A primeira abordagem a se pensar é, porque não fazer um loop de $[l, r]$ somando o valor de cada elemento de cada elemento.

```
while(q--){  
    int l,r; cin >> l >> r;  
    int sum = 0;  
    for(int i = l; i ≤ r;i++) sum += a[i];  
    cout << sum << endl;  
}
```

Perceba que se em cada consulta ele pedir um intervalo grande, perto do tamanho do array a complexidade de uma consulta fica $O(N)$, como são Q consultas a complexidade final fica $O(NQ)$ que dá $4 * 10^{10}$ operações

Problema Simplificado

Problema Simplificado

Imagine que agora nas consultas, todo l tem valor 1, ou seja queremos calcular valores do prefixo.

Problema Simplificado

Imagine que agora nas consultas, todo l tem valor 1, ou seja queremos calcular valores do prefixo.

Nesse caso poderíamos precalcular o valor de cada soma de prefixo apenas com um loop.

Problema Simplificado

Imagine que agora nas consultas, todo l tem valor 1, ou seja queremos calcular valores do prefixo.

Nesse caso poderíamos precalcular o valor de cada soma de prefixo apenas com um loop.

```
vector<int> soma_pref(n + 1);  
for(int i = 1; i ≤ n; i++){  
    soma_pref[i] = soma_pref[i - 1] + a[i - 1];  
}
```

Problema Simplificado

Imagine que agora nas consultas, todo l tem valor 1, ou seja queremos calcular valores do prefixo.

Nesse caso poderíamos precalcular o valor de cada soma de prefixo apenas com um loop.

```
vector<int> soma_pref(n + 1);  
for(int i = 1; i ≤ n; i++){  
    soma_pref[i] = soma_pref[i - 1] + a[i - 1];  
}
```

Perceba que o array que guarda a soma está 1 indexado, isso é útil porque agora na posição 0 nós guardamos o prefixo vazio.

Problema Simplificado

Imagine que agora nas consultas, todo l tem valor 1, ou seja queremos calcular valores do prefixo.

Nesse caso poderíamos precalcular o valor de cada soma de prefixo apenas com um loop.

```
vector<int> soma_pref(n + 1);  
for(int i = 1; i ≤ n; i++){  
    soma_pref[i] = soma_pref[i - 1] + a[i - 1];  
}
```

Perceba que o array que guarda a soma está 1 indexado, isso é útil porque agora na posição 0 nós guardamos o prefixo vazio.

Então para responder a $querie[l, r]$, nós já temos o valor salvo em $soma_pref[r]$, basta imprimir;

Voltando ao problema original

Voltando ao problema original

A observação que precisamos ter aqui é que qualquer range l, r pode ser representado como a subtração de dois prefixos do array da soma de prefixos, mais especificamente:

Voltando ao problema original

A observação que precisamos ter aqui é que qualquer range l, r pode ser representado como a subtração de dois prefixos do array da soma de prefixos, mais especificamente:

soma de $[l, r] = \text{somapref}[r] - \text{somapref}[l - 1]$.

Voltando ao problema original

A observação que precisamos ter aqui é que qualquer range l, r pode ser representado como a subtração de dois prefixos do array da soma de prefixos, mais especificamente:

soma de $[l, r] = \text{somapref}[r] - \text{somapref}[l - 1]$.

A complexidade para acessar duas vezes um array é $O(1)$ então conseguimos resolver o problema em $O(Q)$ ou melhor dizendo $O(N + Q)$.

Voltando ao problema original

A observação que precisamos ter aqui é que qualquer range l, r pode ser representado como a subtração de dois prefixos do array da soma de prefixos, mais especificamente:

soma de $[l, r] = \text{somapref}[r] - \text{somapref}[l - 1]$.

A complexidade para acessar duas vezes um array é $O(1)$ então conseguimos resolver o problema em $O(Q)$ ou melhor dizendo $O(N + Q)$.

Mas e se l for igual a 1? Nesse caso nós subtrairíamos $\text{somapref}[0]$ que é o prefixo vazio.

Soma de prefixos e Busca binária

Soma de prefixos e Busca binária

Se um array A é formado inteiros não negativos, então a soma de prefixos dele pode ser visualizada como uma função monotônica, logo, podemos fazer uma busca binária.

Soma de prefixos e Busca binária

Se um array A é formado inteiros não negativos, então a soma de prefixos dele pode ser visualizada como uma função monotônica, logo, podemos fazer uma busca binária.

Aplicações:

Soma de prefixos e Busca binária

Se um array A é formado inteiros não negativos, então a soma de prefixos dele pode ser visualizada como uma função monotônica, logo, podemos fazer uma busca binária.

Aplicações:

- Qual o menor prefixo que tem soma $\geq K$.
- Para uma posição r , qual o menor l ($l < r$) tal que a soma $[l, r] \leq X$
- Contar quantos subarrays tem soma $\geq S$

Soma de prefixos e Busca binária

Se um array A é formado inteiros não negativos, então a soma de prefixos dele pode ser visualizada como uma função monotônica, logo, podemos fazer uma busca binária.

Aplicações:

- Qual o menor prefixo que tem soma $\geq K$.
- Para uma posição r , qual o menor l ($l < r$) tal que a soma $[l, r] \leq X$
- Contar quantos subarrays tem soma $\geq S$

Questão

Problema clássico

Problema clássico

Dado um array A , quantos subarrays tem soma exatamente igual a K .

Problema clássico

Dado um array A , quantos subarrays tem soma exatamente igual a K .

Problema

Problema clássico

Dado um array A , quantos subarrays tem soma exatamente igual a K .

Problema

A ideia principal desse problema vem a partir de uma manipulação matemática.

Problema clássico

Dado um array A, quantos subarrays tem soma exatamente igual a K.

Problema

A ideia principal desse problema vem a partir de uma manipulação matemática.

$$somapref[r] - somapref[l - 1] = K$$

Problema clássico

Dado um array A, quantos subarrays tem soma exatamente igual a K.

Problema

A ideia principal desse problema vem a partir de uma manipulação matemática.

$$somapref[r] - somapref[l - 1] = K$$

$$somapref[r] - K = somapref[l - 1]$$

Problema clássico

Dado um array A, quantos subarrays tem soma exatamente igual a K.

Problema

A ideia principal desse problema vem a partir de uma manipulação matemática.

$$somapref[r] - somapref[l - 1] = K$$

$$somapref[r] - K = somapref[l - 1]$$

Para cada r queremos achar quantos l 's existem tal que $somapref[r] - K = somapref[l - 1]$. A soma disso para todos os r 's será a resposta final , o algoritmo consiste em iterar o array de forma crescente e salvar quantas vezes um valor já apareceu antes.

Problema clássico

Dado um array A, quantos subarrays tem soma exatamente igual a K.

Problema

A ideia principal desse problema vem a partir de uma manipulação matemática.

$$somapref[r] - somapref[l - 1] = K$$

$$somapref[r] - K = somapref[l - 1]$$

Para cada r queremos achar quantos l 's existem tal que $somapref[r] - K = somapref[l - 1]$. A soma disso para todos os r 's será a resposta final, o algoritmo consiste em iterar o array de forma crescente e salvar quantas vezes um valor já apareceu antes.

```
for(int x : arr){  
    int soma_r += x;  
    resposta += cnt[soma_r - k] // quantas vezes apareceu um l com somapref[l - 1] = somapref[r] - K  
    cnt[soma_r]++; // esse cara se torna uma opção para quem está na frente dele  
}
```

Soma num Intervalo

Soma num Intervalo

Problema Motivador

Soma num Intervalo

Problema Motivador

Dado um array com N inteiros, Processe Q consultas na forma:

Soma num Intervalo

Problema Motivador

Dado um array com N inteiros, Processe Q consultas na forma:

" \$l, r, x \$ Some X em todos os elementos no intervalo $[l, r]$ "

Soma num Intervalo

Problema Motivador

Dado um array com N inteiros, Processe Q consultas na forma:

" \$l, r, x\$ Some X em todos os elementos no intervalo $[l, r]$ "

No final imprima o array após todas as somas

Soma num Intervalo

Problema Motivador

Dado um array com N inteiros, Processe Q consultas na forma:

" \$l, r, x\$ Some X em todos os elementos no intervalo $[l, r]$ "

No final imprima o array após todas as somas

Exemplo prático

Soma num Intervalo

Problema Motivador

Dado um array com N inteiros, Processe Q consultas na forma:

" \$l, r, x\$ Some X em todos os elementos no intervalo $[l, r]$ "

No final imprima o array após todas as somas

Exemplo prático

Índice (i)	1	2	3	4	5	6	7	8
Valor	3	2	4	5	1	1	5	3

Soma num Intervalo

Problema Motivador

Dado um array com N inteiros, Processe Q consultas na forma:

" \$l, r, x\$ Some X em todos os elementos no intervalo $[l, r]$ "

No final imprima o array após todas as somas

Exemplo prático

Índice (i)	1	2	3	4	5	6	7	8
Valor	3	2	4	5	1	1	5	3

Consultas

Soma num Intervalo

Problema Motivador

Dado um array com N inteiros, Processe Q consultas na forma:

"\$l, r, x\$ Some X em todos os elementos no intervalo $[l, r]$ "

No final imprima o array após todas as somas

Exemplo prático

Índice (i)	1	2	3	4	5	6	7	8
Valor	3	2	4	5	1	1	5	3

Consultas

- $[2, 4, 2]$
- $[5, 6, 1]$

Força bruta

Força bruta

A primeira abordagem a se pensar é, porque não fazer um loop de $[l, r]$ somando x em todos os elementos

Força bruta

A primeira abordagem a se pensar é, porque não fazer um loop de $[l, r]$ somando x em todos os elementos

```
while(q--){  
    int l,r,x; cin >> l >> r >> x;  
    for(int i = l; i ≤ r;i++) a[i] += x;;  
    cout << sum << endl;  
}
```

Força bruta

A primeira abordagem a se pensar é, porque não fazer um loop de $[l, r]$ somando x em todos os elementos

```
while(q--){  
    int l,r,x; cin >> l >> r >> x;  
    for(int i = l; i ≤ r;i++) a[i] += x;;  
    cout << sum << endl;  
}
```

Perceba que se em cada querie ele pedir um intervalo grande, perto do tamanho do array a complexidade de uma consulta fica $O(N)$, como são Q consultas a complexidade final fica $O(NQ)$ que dá $4 * 10^{10}$ operações

Array de diferenças

Array de diferenças

Tentaremos calcular o array de diferenças (AD) que guarda para cada índice o quanto seu valor foi alterado juntando todas as consultas.

Array de diferenças

Tentaremos calcular o array de diferenças (AD) que guarda para cada índice o quanto seu valor foi alterado juntando todas as consultas.

Considere um Array de tamanho 5 , inicialmente o AD está assim 0,0,0,0,0,0 , o array tem um índice a mais propositalmente.

Array de diferenças

Tentaremos calcular o array de diferenças (AD) que guarda para cada índice o quanto seu valor foi alterado juntando todas as consultas.

Considere um Array de tamanho 5 , inicialmente o AD está assim 0,0,0,0,0,0 , o array tem um índice a mais propositalmente.

Agora, pensando na soma de prefixos o que acontece quando somamos 1 do índice 2 até o índice 4.

Array de diferenças

Tentaremos calcular o array de diferenças (AD) que guarda para cada índice o quanto seu valor foi alterado juntando todas as consultas.

Considere um Array de tamanho 5 , inicialmente o AD está assim 0,0,0,0,0,0 , o array tem um índice a mais propositalmente.

Agora, pensando na soma de prefixos o que acontece quando somamos 1 do índice 2 até o índice 4.

\$ 0,1,0,0,-1,0\$, perceba que quando calcularmos a soma de prefixos todos os valores entre 2 e 4 vão ser somados em 1, essa soma se propaga. Porém ao chegarmos no índice 5 esse valor é subtraído o valor para de ser propagado.

Array de diferenças

Tentaremos calcular o array de diferenças (AD) que guarda para cada índice o quanto seu valor foi alterado juntando todas as consultas.

Considere um Array de tamanho 5 , inicialmente o AD está assim 0,0,0,0,0,0 , o array tem um índice a mais propositalmente.

Agora, pensando na soma de prefixos o que acontece quando somamos 1 do índice 2 até o índice 4.

\$ 0,1,0,0,-1,0\$, perceba que quando calcularmos a soma de prefixos todos os valores entre 2 e 4 vão ser somados em 1, essa soma se propaga. Porém ao chegarmos no índice 5 esse valor é subtraído o valor para de ser propagado.

$somapref = [0, 1, 1, 1, 0, 0]$, após calcularmos a soma de prefixos teremos o array final.

Problema

Problema

Link

Problema

Link

```
vector<int> ps(n + 1);
while (q--)
{
    int l, r;
    cin >> l >> r;
    l--, r--;
    ps[l]++;
    ps[r + 1]--;
}

for (int i = 1; i ≤ n; i++)
{
    ps[i] += ps[i - 1];
}
```


Problema

Link

```
vector<int> ps(n + 1);
while (q--)
{
    int l, r;
    cin >> l >> r;
    l--, r--;
    ps[l]++;
    ps[r + 1]--;
}

for (int i = 1; i ≤ n; i++)
{
    ps[i] += ps[i - 1];
}
```

ps guarda o array após todas as alterações.

Referências

- Documentação C++
- Curso Busca Binária
- Aula Busca Binária Errichto
- Aula Soma de Prefixos Errichto
- Array de Diferenças
- Livro CPH

Contatos

Entre no discord para dúvidas



Entre no grupo de questões



Obrigado por acompanhar a aula

Por favor preencha o formulário de feedback com sugestões e críticas para a próxima aula 🙏.

